Analítica de Datos

Clase 11. Selección de variables e introducción a redes neuronales

Cómo elegir los predictores y comprender el aprendizaje de una red

Luis Miguel Gómez Meneses
Universidad Cooperativa de Colombia
Ingeniería de Software — Séptimo semestre

30 de octubre de 2026

Duración de la sesión: 3 horas

¿Qué son las características?

En un conjunto de datos tabular distinguimos **registros** y **características**.

	Características: columnas					
Registros	A	B	C	D	E	F
Registro 1	...	...	...	...	...	...
Registro 2	...	...	...	...	...	...
Registro 3	...	...	...	...	...	...

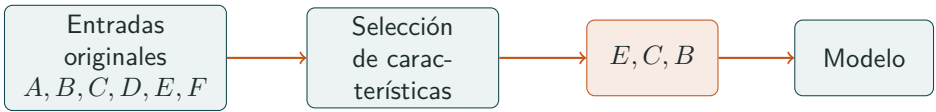
Ejemplo: información de clientes

Cada fila puede representar un cliente. Sus características pueden incluir **edad, ingresos, deuda y valor de la hipoteca**.

La variable objetivo es aquello que queremos predecir; se distingue de las características utilizadas como entradas.

Seleccionar características: reducir las entradas

Partimos de las características disponibles y buscamos un subconjunto útil para realizar la predicción.



¿Qué buscamos?

Conservar información relevante y reducir entradas innecesarias o redundantes. En este ejemplo pasamos de seis características a tres.

Pregunta: ¿Tener más columnas garantiza mejores predicciones?

Adaptado de Codificando Bits: Selección de características en Machine Learning.

¿Por qué seleccionar? Dimensionalidad y relevancia

Al aumentar las dimensiones, los datos pueden quedar más dispersos y se necesitan más observaciones para reconocer patrones con fiabilidad.

Ejemplo del material: riesgo cardíaco

Se proponen como entradas:

$$\{\text{edad, altura, peso, glucosa, deuda}\}$$

El modelo busca estimar la probabilidad de una enfermedad cardíaca.

Pregunta sobre la pertinencia de las entradas

El ejemplo propone revisar si la deuda aporta información útil al problema, frente a las variables biológicas. Su exclusión debe justificarse con el contexto y los datos.

Para discutir: ¿Qué evidencia necesitaríamos para decidir si una característica aporta a la predicción?

Adaptado de Codificando Bits: Selección de características en Machine Learning.

¿Por qué seleccionar? Eficiencia computacional

Reducir las entradas puede disminuir el trabajo necesario para entrenar el modelo y realizar predicciones.

Más características

Más información que procesar
Posible mayor costo de cálculo

Subconjunto seleccionado

Menos entradas que procesar
Posible reducción del costo

Dos momentos del proceso

Entrenamiento: aprender a partir de los datos.

Inferencia: utilizar el modelo para generar predicciones.

La reducción de tiempo y complejidad depende del algoritmo. Menos entradas no siempre implica menos parámetros.

Adaptado de Codificando Bits: Selección de características en Machine Learning.

¿Por qué seleccionar? Reducir el riesgo de sobreajuste

Sobreajuste

El modelo obtiene buenos resultados con los datos de entrenamiento, pero pierde desempeño con datos nuevos.

	Entrenamiento	**Validación**
Situación con sobreajuste	Desempeño alto	Desempeño mucho menor
Situación deseable	Buen desempeño	Buen desempeño, cercano al de entrenamiento

Relación con las características

Las entradas irrelevantes pueden facilitar que el modelo aprenda patrones accidentales del entrenamiento. Retirarlas puede ayudar a generalizar.

¿Cuándo utilizamos selección de características?

Datos tabulares y modelos clásicos

Es habitual seleccionar columnas antes de construir modelos como:

- Regresión lineal.
- Máquinas de soporte vectorial.
- Árboles de decisión.
- Random Forest.

Datos no estructurados y aprendizaje profundo

En imágenes, audio o texto, las redes profundas suelen aprender representaciones durante el entrenamiento. No se aplica necesariamente la misma selección manual de columnas.

Una red neuronal que recibe datos tabulares también puede utilizar un subconjunto seleccionado de variables.

Adaptado de Codificando Bits: Selección de características en Machine Learning.

Tres familias de métodos

Filtrado

Puntúa las características mediante medidas estadísticas, sin entrenar el modelo predictivo final.

Wrapper

Compara el desempeño de un modelo al entrenarlo con diferentes subconjuntos de características.

Embebidos

La identificación de características útiles forma parte del entrenamiento del modelo.

Idea para recordar: La diferencia está en cómo se decide qué entradas conservar.

Adaptado de Codificando Bits: Selección de características en Machine Learning.

Filtrado: puntuar, ordenar y seleccionar

Se asigna un puntaje a cada característica mediante un criterio estadístico, como correlación o varianza.

Característica	A	B	C	D	E	F
Puntaje	0,68	0,81	0,87	0,25	0,92	0,72

Ejemplo del material: seleccionar las mejores tres

Ordenamos los puntajes de mayor a menor:

$$E > C > B > F > A > D$$

Con $k = 3$, seleccionamos:

$$\boxed{\{E, C, B\}}$$

Estas tres características se utilizan como entradas del modelo.

Los valores son los puntajes del ejemplo original; no se especifica que sean coeficientes de Pearson. El filtro se ajusta con los datos de entrenamiento.

Wrapper: el desempeño del modelo guía la selección

Procedimiento

Elegimos un subconjunto, entrenamos el modelo y evaluamos su desempeño. Repetimos con otros subconjuntos.

Hacia adelante

Comenzamos sin características y agregamos una en cada iteración.

$$0 \to 1 \to 2 \to 3$$

Hacia atrás

Comenzamos con todas las características y retiramos una en cada iteración.

$$6 \to 5 \to 4 \to 3$$

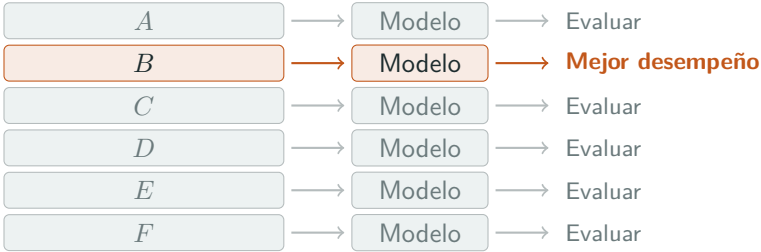
Usamos el mismo algoritmo, hiperparámetros, particiones y métrica para comparar los candidatos.

Las decisiones utilizan validación. La prueba final queda reservada para la evaluación final.

Adaptado de Codificando Bits: Selección de características en Machine Learning.

Hacia adelante: primero elegimos una característica

Entrenamos seis modelos, cada uno con una entrada. En el ejemplo, **B obtiene el mejor desempeño**.

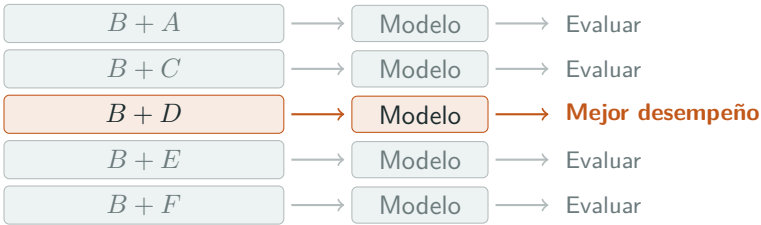


Selección provisional: $\boxed{\{B\}}$.

Adaptado de Codificando Bits: Selección de características en Machine Learning.

Hacia adelante: añadimos una segunda característica

Conservamos B y la combinamos con cada entrada restante. Entrenamos cinco nuevos modelos.



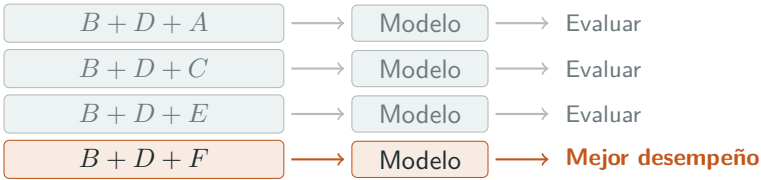
Resultado del ejemplo

La combinación ganadora es $\{B, D\}$ y mejora el desempeño obtenido con B.

Adaptado de Codificando Bits: Selección de características en Machine Learning.

Hacia adelante: seleccionamos la tercera característica

Conservamos B y D. Probamos añadir A, C, E o F.



Criterio de parada del ejemplo

Se fijó un máximo de tres características. La selección termina en:

$$\boxed{\{B, D, F\}}$$

También puede detenerse el recorrido cuando la mejora sea insuficiente. No se garantiza el mejor subconjunto global.

Adaptado de Codificando Bits: Selección de características en Machine Learning.

Hacia atrás: comenzamos con todas las características

Primero entrenamos y evaluamos un modelo con las seis entradas.



¿Qué haremos después?

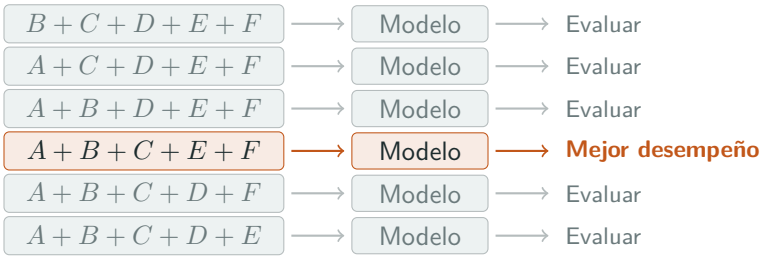
Retiraremos una característica por vez, entrenaremos los modelos candidatos y compararemos su desempeño con esta referencia.

Pregunta: Con seis características, ¿cuántos candidatos de cinco características podemos construir retirando una?

Adaptado de Codificando Bits: Selección de características en Machine Learning.

Hacia atrás: de seis características a cinco

Probamos las seis posibles retiradas. En el ejemplo, eliminar **D** produce la mejor alternativa.

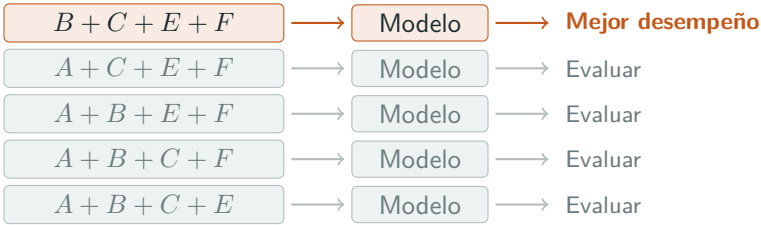


Aceptamos $\boxed{\{A, B, C, E, F\}}$, porque mejora el desempeño de la iteración anterior.

Adaptado de Codificando Bits: Selección de características en Machine Learning.

Hacia atrás: de cinco características a cuatro

Partimos de $\{A, B, C, E, F\}$. Probamos retirar una de sus cinco características.



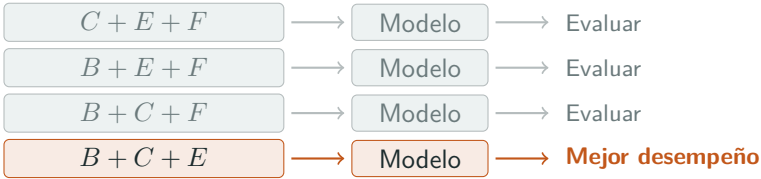
Resultado del ejemplo

Retiramos A y conservamos $\boxed{\{B, C, E, F\}}$, con una mejora frente a la selección anterior.

Adaptado de Codificando Bits: Selección de características en Machine Learning.

Hacia atrás: de cuatro características a tres

Partimos de $\{B, C, E, F\}$. Probamos las cuatro posibles retiradas.



Resultado y parada

Retiramos F y conservamos $\boxed{\{B, C, E\}}$. Como el ejemplo fija un mínimo de tres características, detenemos el recorrido.

Hacia adelante obtuvimos B,D,F; hacia atrás, B,C,E. Los recorridos pueden producir selecciones diferentes.

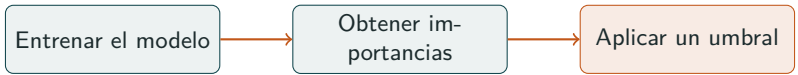
Adaptado de Codificando Bits: Selección de características en Machine Learning.

Métodos embebidos: aprovechar lo que aprende el modelo

En estos métodos, la identificación de entradas útiles está integrada en el entrenamiento.

Ejemplo: árboles de decisión y Random Forest

Durante su construcción, los árboles eligen variables para dividir los datos. A partir de esas divisiones pueden calcularse importancias de las características.



$$\text{Conservar } X_j \quad \text{si} \quad I_j \geq \tau$$

I_j: importancia de la característica. τ: umbral elegido.

Comprobamos después el desempeño del modelo reducido. **La importancia no elimina automáticamente las columnas.**

Adaptado de Codificando Bits: Selección de características en Machine Learning.

Laboratorio: selección de variables con Random Forest

Objetivo

Comparar un modelo con seis variables frente a modelos con solo tres variables seleccionadas.

Datos simulados: seis entradas (A, B, C, D, E y F) y una salida numérica.

Aplicaremos tres tipos de selección:

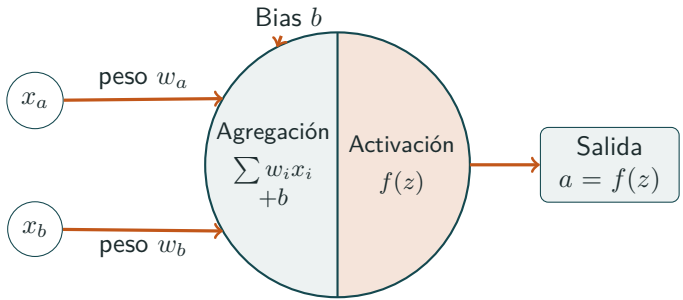
1. **Filtrado:** elegir las tres variables con mayor correlación absoluta con la salida.
2. **Wrapper:** probar subconjuntos con Random Forest mediante selección hacia adelante y hacia atrás.
3. **Basada en el modelo:** elegir las tres variables con mayor importancia en el Random Forest entrenado.

Evaluación: compararemos el RMSE. Un valor menor indica menor error.

Pregunta: ¿Podemos conservar un desempeño similar con menos variables?

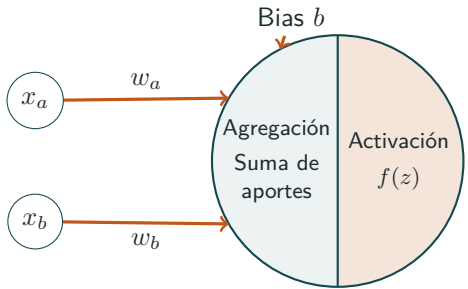
La neurona artificial

Una **neurona artificial** combina las entradas y transforma el resultado para producir una salida. Los **pesos** regulan el aporte de cada entrada y el **bias** desplaza la suma.



Pesos y bias

Los **pesos** multiplican las entradas. El **bias** añade un desplazamiento a la suma. Ambos se ajustan al entrenar.



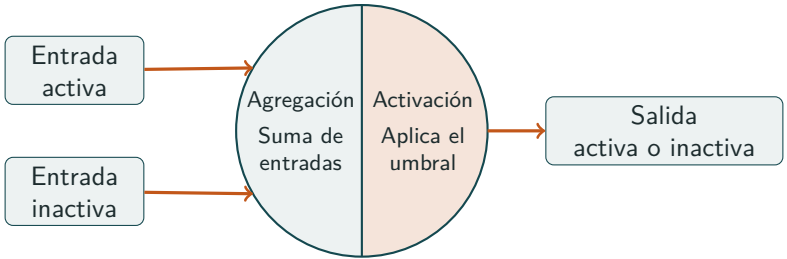
Peso
Regula la magnitud y el signo del aporte.

Bias
Desplaza el resultado de la agregación.

El bias permite desplazar la respuesta incluso cuando las entradas son nulas.

McCulloch y Pitts: activación por umbral

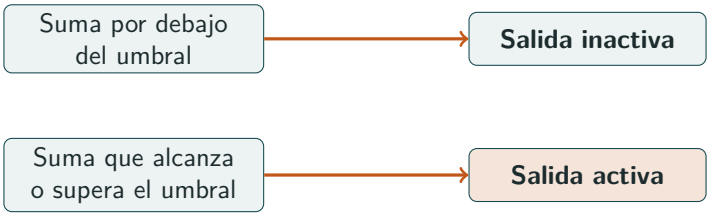
McCulloch y Pitts propusieron un modelo lógico de neurona con entradas y salida binarias: **activas o inactivas**.



Versión simplificada: usamos entradas excitatorias y un umbral fijo. Este modelo aplica una regla lógica y no aprende mediante backward.

El umbral determina la activación

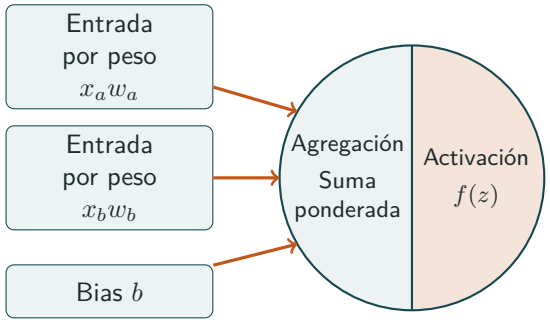
El **umbral** es el nivel que debe alcanzar la suma para activar la salida.



Un umbral más alto exige una suma mayor para activar la neurona.

Agregación: multiplicar y sumar

La **función de agregación** reúne los aportes de las entradas ponderadas y añade el bias.



El resultado agregado

z

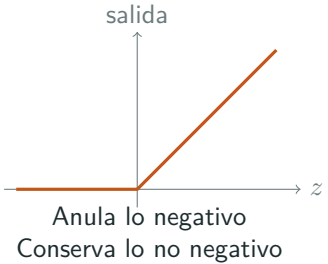
pasa a la activación.

Cada entrada participa según su peso. El bias se suma al conjunto de aportes.

Activación: transformar la suma en una salida

La **función de activación** transforma el resultado de la agregación. Con **ReLU**, las entradas negativas producen una salida nula y las no negativas conservan su valor.

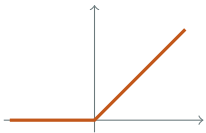




La activación determina la respuesta de la neurona ante la suma recibida.

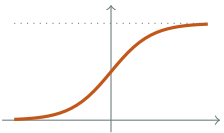
Activaciones y sus usos habituales

ReLU



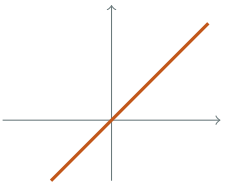
Capas ocultas
Salida no negativa

Sigmoide



Clasificación binaria
Salida acotada

Lineal

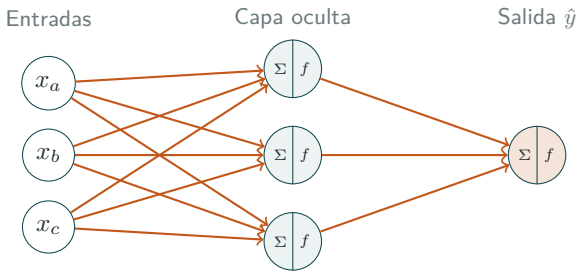


Salida de regresión
Conserva el valor recibido

Las activaciones no lineales permiten aprender relaciones más complejas. La sigmoide suele usarse como salida para estimar una probabilidad.

Una red neuronal conecta varias neuronas

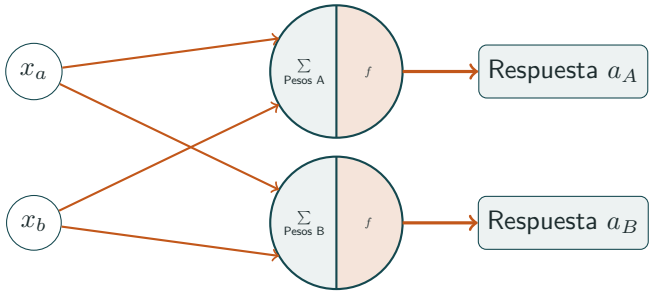
Una **red neuronal** conecta unidades de cálculo organizadas en capas. Las salidas de unas neuronas se convierten en entradas de otras.



Las conexiones tienen pesos. Cada neurona combina sus entradas y aplica una activación.

Cada neurona oculta aprende una combinación propia

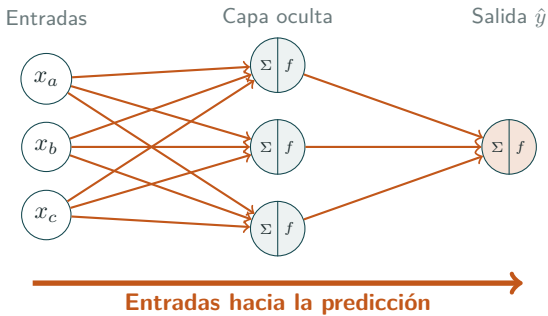
Las neuronas pueden recibir las mismas entradas y producir respuestas diferentes.



Cada neurona tiene sus propios pesos y bias y aprende una representación diferente de las entradas.

Forward: la información avanza hasta la predicción

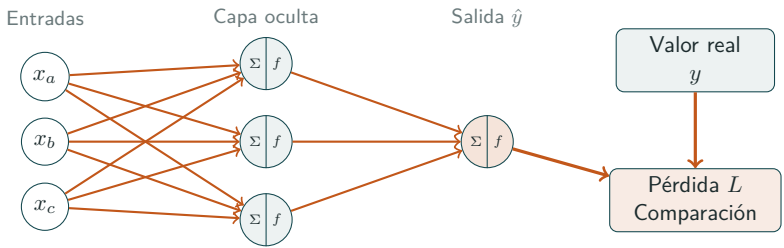
El **forward** calcula la salida recorriendo la red de izquierda a derecha con los pesos y bias actuales.



Durante este recorrido se utilizan los parámetros actuales para obtener la predicción.

Cálculo del error: comparar predicción y valor real

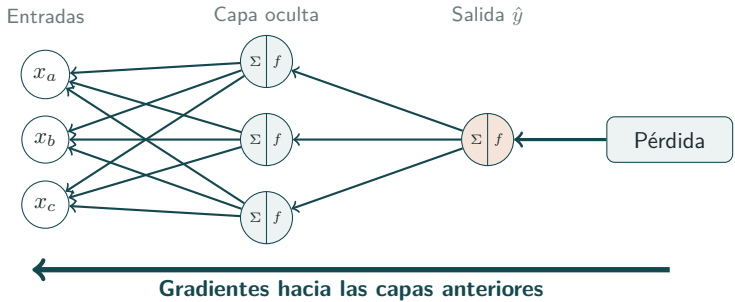
La **pérdida** mide cuánto se aleja la predicción del valor real.



Una pérdida menor indica un mejor ajuste a los valores reales según el criterio elegido.

Backward: los gradientes recorren la red hacia atrás

El **backward** parte de la pérdida y calcula cómo influye cada peso y bias en ella. El **gradiente** indica la dirección y la sensibilidad de la pérdida ante cambios en los parámetros.



El optimizador utiliza los gradientes para ajustar los pesos y los bias.

Actualizar un peso: valor anterior, tasa y gradiente

En **descenso de gradiente**, el nuevo peso se calcula así:

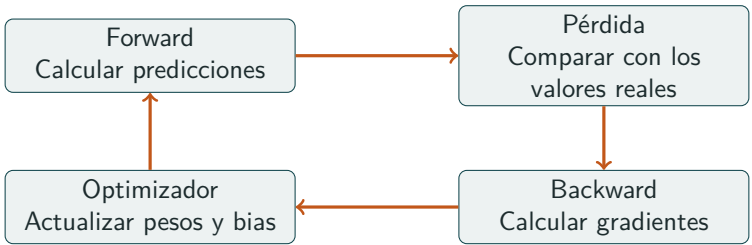
$$w_{\text{nuevo}} = w_{\text{anterior}} - \eta \times g$$

w_{nuevo}	$=$	w_{anterior}	$-$	η	$\times$	g
Peso actualizado		Valor actual		Tasa de aprendizaje		Gradiente $\partial L/\partial w$

La **tasa de aprendizaje** regula el tamaño del ajuste. El **gradiente** indica cómo cambia la pérdida al cambiar el peso.

Restar el gradiente orienta el ajuste hacia una disminución local de la pérdida. El bias se actualiza de forma análoga con su propio gradiente.

Entrenamiento: repetir el ciclo con nuevos lotes



El entrenamiento repite este ciclo sobre los lotes. Una vez entrenada la red, el forward permite obtener predicciones con los parámetros aprendidos.

Parámetros e hiperparámetros

Parámetros

Pesos y bias

Los ajusta el entrenamiento

Hiperparámetros

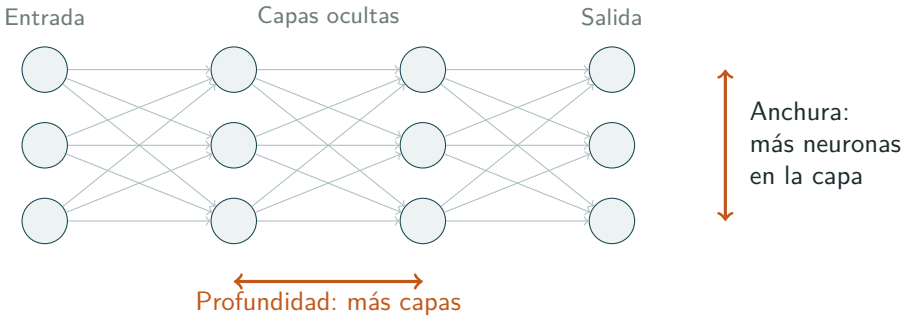
Capas, neuronas, activaciones
Tasa, épocas y tamaño del lote

Los configuramos y validamos

La validación permite comparar configuraciones y elegir las que funcionan mejor en datos que no se usaron para ajustar los parámetros.

Arquitectura: profundidad y anchura de la red

La **profundidad** corresponde a la cantidad de capas. La **anchura** corresponde a la cantidad de neuronas en una capa.

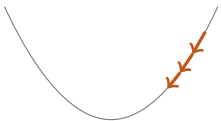


Aumentar la capacidad de la red también puede aumentar el riesgo de sobreajuste.

Tasa de aprendizaje: cuánto avanzamos en cada ajuste

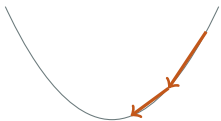
La tasa regula la magnitud del ajuste junto con el gradiente y el optimizador.

Pequeña



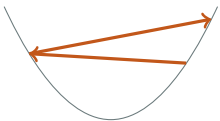
Avances lentos

Adecuada



Avances útiles

Muy grande

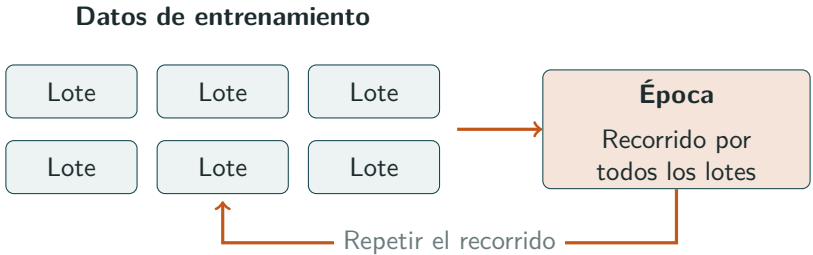


Puede oscilar

Esquema conceptual: altura = pérdida; posición horizontal = valor de un peso.

Épocas y lotes: organizar el entrenamiento

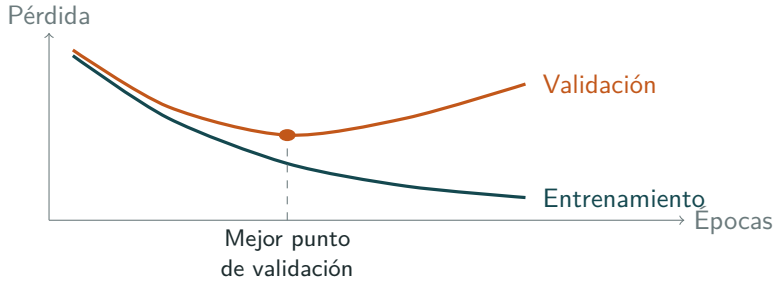
Una **época** es un recorrido completo por los datos de entrenamiento. Un **lote** es un grupo de registros usado para calcular una actualización.



El tamaño del lote y la cantidad de épocas son hiperparámetros. Los lotes de validación se usan para evaluar, sin actualizar los parámetros.

La validación ayuda a decidir cuándo detenernos

El error de entrenamiento puede bajar mientras el de validación empieza a subir.

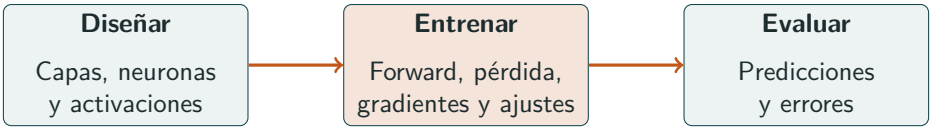


Curvas ilustrativas. Early stopping permite parar si la validación deja de mejorar.

Podemos conservar los parámetros del mejor punto de validación.

TensorFlow: nuestra herramienta para diseñar redes

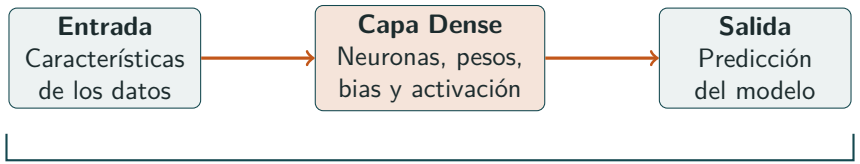
TensorFlow será la herramienta que utilizaremos en Python para **diseñar, entrenar y evaluar redes neuronales**.



Usaremos **Keras**, mediante `tf.keras`, para construir la red con pocas instrucciones.

Keras: construir la red por capas

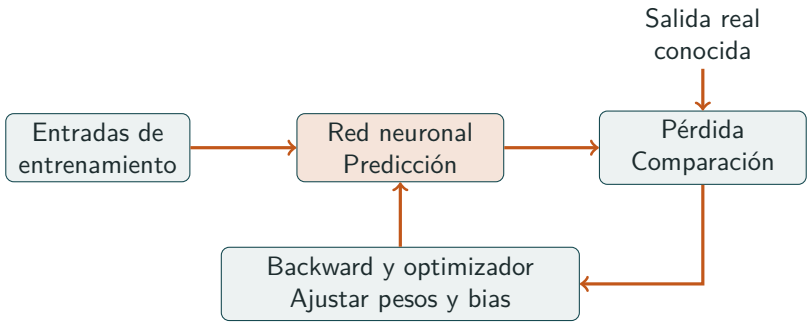
Keras ofrece una interfaz para definir las capas y organizar el entrenamiento. La usaremos mediante `tf.keras`.



En una capa `Dense`, cada neurona recibe las salidas de la capa anterior. Configuramos la arquitectura y el entrenamiento; TensorFlow ejecuta los cálculos.

Entrenamiento supervisado de una red neuronal

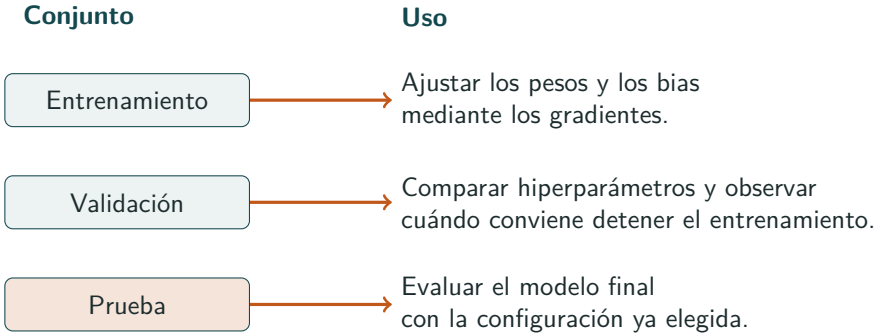
Entrenar consiste en ajustar los pesos y los bias usando ejemplos con una salida conocida. Buscamos que la red aprenda patrones útiles para datos nuevos.



Generalizar significa conservar un buen desempeño en ejemplos que no se utilizaron para ajustar los parámetros.

Entrenamiento, validación y prueba

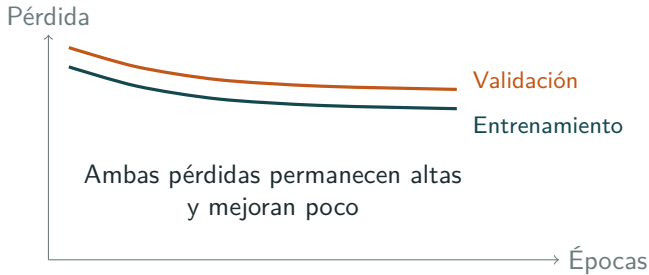
Separamos los datos para distinguir el aprendizaje de la evaluación.



Validación y prueba no actualizan los pesos. La prueba se reserva para la evaluación final, sin usarla para elegir hiperparámetros.

Subajuste: aprendizaje insuficiente

Hay **subajuste** cuando la red no aprende suficientemente los patrones y su desempeño es bajo tanto en entrenamiento como en validación.

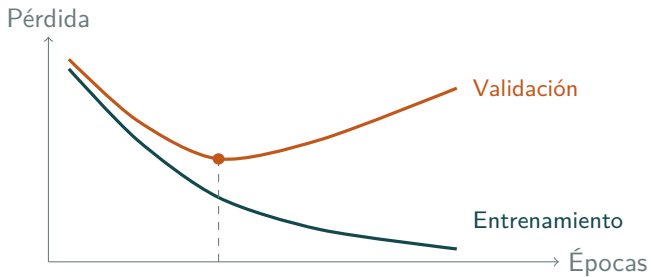


Posibles causas: poca capacidad, entrenamiento insuficiente o una configuración que dificulta aprender.

Curvas conceptuales. Una brecha pequeña no basta si el desempeño es bajo.

Sobreajuste: aprender particularidades del entrenamiento

Hay **sobreajuste** cuando la red aprende detalles del entrenamiento que no se trasladan bien a nuevos datos.

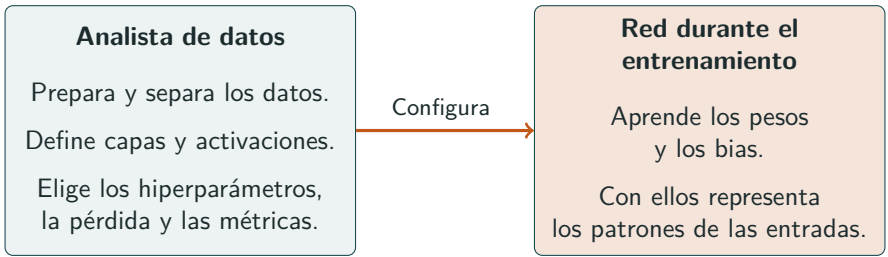


La pérdida de entrenamiento baja mientras la de validación aumenta. Puede aparecer con alta capacidad, pocos ejemplos o entrenamiento prolongado.

Curvas conceptuales. Un buen resultado en entrenamiento no garantiza generalización.

Decisiones del analista y aprendizaje de la red

El analista configura el proceso. Durante el entrenamiento, el optimizador ajusta los parámetros de la red a partir de los datos.



En este laboratorio la arquitectura permanece fija durante cada entrenamiento. Las entradas y las clases del problema determinan el formato de entrada y salida.

Hiperparámetros que configura el analista

Los hiperparámetros definen la estructura y la forma de aprender. Se comparan utilizando validación.

Configuración	**Aspecto que controla**
Capas y neuronas ocultas	Capacidad de representar patrones.
Activaciones	Transformación aplicada por las neuronas.
Optimizador y tasa de aprendizaje	Regla de actualización y escala de los ajustes.
Épocas y tamaño del lote	Recorridos por los datos y registros por actualización.
Tasa de Dropout	Proporción de activaciones anuladas temporalmente.
Paciencia de parada temprana	Espera sin mejora antes de detener el entrenamiento.

Cambiar una configuración a la vez facilita interpretar su efecto. Una red más grande o más épocas no garantizan mejores predicciones.

Dropout y parada temprana frente al sobreajuste

Son estrategias que probaremos para favorecer la generalización. Su efecto se comprueba con la validación.

Dropout

Anula aleatoriamente algunas activaciones durante el entrenamiento y reescala las restantes.

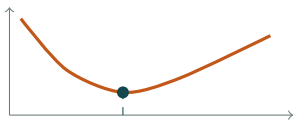


Máscara temporal
sobre las activaciones

No elimina neuronas permanentemente. Se desactiva al evaluar y predecir.

Parada temprana: EarlyStopping

Vigila la pérdida de validación y detiene el entrenamiento cuando deja de mejorar durante la paciencia definida.



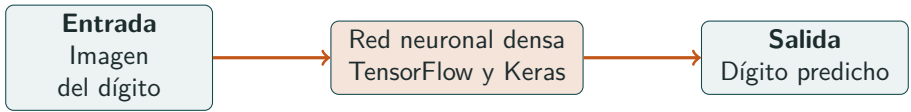
Mejor validación

En el laboratorio restauraremos los pesos del mejor punto de validación.

Laboratorio 23: clasificación de dígitos con MNIST

Objetivo

Entrenar una red neuronal, interpretar sus predicciones y observar cómo la configuración influye en el subajuste y el sobreajuste.



Experimentación: modelo de referencia, configuraciones de subajuste y sobreajuste, Dropout, parada temprana y comparación de tasas de aprendizaje.

Evaluación: curvas de pérdida y exactitud, predicciones y matriz de confusión. La prueba final corresponde al modelo de referencia.

Desarrollo: notebook con celdas de Markdown y código. Las conclusiones se basan en los resultados obtenidos al ejecutarlo.